# Recreate ragcap from zero

Written for: an engineer who has never seen this project and wants to rebuild it, run every scenario, and check the result against the recorded fingerprints. Time: about 15 minutes plus model downloads. Date of this guide: 2026-10-23.

ragcap reproduces Kumar's "RAG Capstone" design (LangGraph router, document RAG with grade and rewrite, SQL agent over uploaded tables, general agent, Postgres + pgvector, FastAPI, Gradio) on local models, and adds two things that design did not have: frozen evaluation sets with quality metrics, and a version-aware filter that keeps superseded documents out of retrieval. Everything below was run on an Apple M5 Pro (15 cores, 24 GB, macOS 26.6) with no Homebrew, no Docker and no admin rights.

## 0. What you need before starting

| Need | Why | Check |
|---|---|---|
| Python 3.12 (arm64) | the venv; `uv python install 3.12` needs no admin | `python3.12 --version` |
| Ollama with `llama3.2:3b` and `nomic-embed-text` | the LLM and the embedding model, local, no API key (2.3 GB disk, ~2.7 GB memory). Optional: `qwen3:8b` (recommended second-opinion grader), `qwen2.5vl:3b` (drawings) | `ollama list` |
| Postgres 16 + pgvector: nothing to install | `pgctl.sh` runs it natively from the pip wheel `pgserver` (data in `.pgdata/`); it would use Docker if the `docker` command existed | `./pgctl.sh status` after 2.1 |
| About 12 GB free disk | models (up to 10.7 GB), packages, database | `df -h ~` |
| Lid open or `caffeinate` | a sleeping Mac pauses a run | every long script uses `caffeinate -i` |

How to install each tool, with download locations: `docs/INSTALL_AND_USE.md`.

## 1. Get the code

`git clone https://github.com/shreenathacc22/ragcap && cd ragcap`, or use the project's `build` folder. Never copy `.venv`, `.pgdata/`, `uploads/` or `traces.jsonl` from another machine: the rebuild makes new ones. A copied `.env` may be kept.

| Path | What it is |
|---|---|
| `ragcap/config.py` | every setting: Postgres DSN from `.env`, model names, `TOP_K=6`, `VERSION_AWARE`, grader switches |
| `ragcap/db.py`, `ragcap/llm.py` | Postgres connection (pgvector registered); OpenAI-compatible client to Ollama; `embed()` |
| `ragcap/ingest.py` | file to text to chunks to vectors; front-matter to metadata; CSV/Excel to SQL tables; drawings through the vision model |
| `ragcap/graph.py` | the LangGraph: router, retrieve, grade, rewrite, generate, sql_agent, generate_sql, general; `ask()` |
| `schema.sql`, `step1_check.py` | tables `documents`, `chunks` (vector(768), HNSW), `uploaded_tables`; the first check |
| `golden/` | `golden.jsonl` (27 cases, tuned on), `golden_heldout.jsonl` (25 unseen), `amendments.jsonl` |
| `eval.py`, `b1_eval.py`, `b1_check.py`, `scenarios.py`, `pid_eval.py`, `summarize_series.py` | answer-level evaluation; retrieval OFF/ON ablation; data checks; the scenario catalogue; P&ID evaluation; series summary |
| `recreate.sh`, `services.sh`, `pgctl.sh`, `run_series.sh` | one-command rebuild; start/stop/status/check; the database; repeated runs |

| Path | What it is |
|---|---|
| `api.py`, `ui.py`, `web/`, `try.py` | FastAPI, Gradio, the static web page, shell probe |
| `data/docs_versioned/` (16 files), `data/tables/broker_inventory.csv` | 8 Kafka runbooks in an old and a current version, and one table |

## 2. The build, step by step

`./recreate.sh` runs 2.1 to 2.6 unattended and checks the fingerprints in section 4. Read this section once so you know what it does.

### 2.1 Settings, Python, database

```
PW=$(openssl rand -hex 12)
printf 'PGHOST=localhost\nPGPORT=5433\nPGUSER=ragcap\nPGPASSWORD=%s\nPGDATABASE=ragcap\n' "$PW" > .env; chmod 600 .env
python3.12 -m venv .venv && .venv/bin/pip install -r requirements.txt -r requirements-local-pg.txt
./pgctl.sh start
./pgctl.sh psql -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

Verify: `./pgctl.sh psql -tAc "select extversion from pg_extension where extname='vector'"` prints `0.6.2`. The server listens on 127.0.0.1:5433 over TCP only: its Unix socket is off because a deep project path exceeds macOS's 103-byte socket limit. pgvector 0.6.2 has no `hnsw.iterative_scan`, so `retrieve()` sets `hnsw.ef_search = 1000` instead. The password is fixed when the database is first created; if you lose `.env`, run `./pgctl.sh reset` and start again.

### 2.2 Schema and first check

Do: `.venv/bin/python step1_check.py`. Expect, as the last lines:

```
tables: ['chunks', 'documents', 'uploaded_tables']
embedding dim: 768
nearest: Kafka consumer lag alert threshold 0.383
STEP 1 OK
```

### 2.3 Ingest the versioned corpus and the table

```
.venv/bin/python -m ragcap.ingest data/docs_versioned
.venv/bin/python -m ragcap.ingest data/tables
./pgctl.sh psql -tAc "select (select count(*) from documents)||' docs, '|| \
  (select count(*) from chunks)||' chunks, '||(select count(*) from documents where superseded)||' superseded, '|| \
  (select count(*) from uploaded_tables)||' tables'"
```

Expect `16 docs, 16 chunks, 8 superseded, 1 tables`. Each runbook starts with a front-matter block (`doc`, `version`, `effective_date`, `superseded`, `status`); it becomes the `documents` row and is removed before embedding, so the word SUPERSEDED never reaches the vector.

### 2.4 Data checks and negative tests

Do: `.venv/bin/python b1_check.py --no-trap --negative` then `.venv/bin/python b1_check.py`. Expect `PASS` both times; all three injected faults `reported` (old version marked current, version stored as text, banner in embedded text); pair similarity old vs current `0.963 / 0.990`; `old version ranked first for 1/8 plain questions`. If a negative test prints `NOT REPORTED`, the checker is broken and nothing downstream can be trusted.

### 2.5 Smoke the graph: one question per route

Do: `.venv/bin/python try.py "Thanks!" "How many brokers are in rack-b?" "At what disk utilisation should a broker page?"`

```
Q: Thanks!                            route=general    path: router > general              ~0.3 s
Q: How many brokers are in rack-b?    route=database   path: router > sql_agent > generate_sql   ~0.7 s
   A: There are 2 brokers in rack-b. This was computed from the broker_inventory table.
Q: At what disk utilisation should ...  route=documents  path: router > retrieve > grade > generate ~0.8-1.5 s
   A: A broker page should be triggered when the disk utilisation exceeds 75%.
```

## 2.6 The retrieval ablation: filter OFF then ON

Do: `.venv/bin/python b1_eval.py --latency 3` (about 20 s; no LLM, deterministic). Expect:

```
OFF: superseded 99/192 (51.6%) | trap 2/12 (stale from old chunk 10, any stale string 10) | control 20/20
ON : superseded 0/192 (0.0%)  | trap 12/12 (stale from old chunk 0, any stale string 6) | control 20/20
latency: off and on 12-13 ms median, never more than 1 ms apart
```

Every run rewrites `results/ablation_summary.md` with that run's timing, so compare OFF with ON within one run, not with a number from another run.

## 2.7 Answer-level evaluation

Do: `caffeinate -i .venv/bin/python eval.py` (27 cases, under a minute) or `--golden golden/golden_heldout.jsonl --tag unseen`. For repeated runs: `./run_series.sh <label> llama3.2:3b` then `.venv/bin/python summarize_series.py <label>`.

Expect (llama3.2:3b, identical over 3 runs): tuned set router 100%, answers 92.6% (doc-015 and db-004 fail), retrieval hit 100%; unseen set router 100%, answers 80.0% (h-doc-01, -05, -13, -14, -15 fail). With the recommended grader (`GRADE_MODEL=qwen3:8b GRADE_MODE=escalate GRADE_MAX_TOKENS=600`): unseen 96.0% (only h-doc-15). Version filter at answer level: OFF 88.9%, ON 92.6%.

## 2.8 API and interfaces

Do: `.venv/bin/uvicorn api:app --port 8000 &` (use `--port 8765` if 8000 is taken: `lsof -nP -iTCP:8000 -sTCP:LISTEN`), then `curl -s localhost:8000/health`; open http://127.0.0.1:8000/ for the web page; `.venv/bin/python ui.py` for Gradio at http://127.0.0.1:7860.

Expect: `/health` returns `{"documents":16,"chunks":16,"tables":["broker_inventory"]}`. `POST /chat` streams one `node` event per graph step, then an `answer` event (route, citations, SQL, total ms). `POST /upload` ingests a file. LangSmith is opt-in (`LANGSMITH_TRACING=true`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT` in `.env`); with a wrong key the question is still answered.

## 2.9 P&ID branch (drawings)

Do: `ollama pull qwen2.5vl:3b` (3.2 GB, once), `.venv/bin/python pid/make_pid.py` (three synthetic drawings, `pid/ground_truth.json`, `pid/golden_pid.jsonl`; deterministic), then `.venv/bin/python pid_eval.py --ingest` (about a minute).

Expect `results/pid_eval_summary.md`: 38 of 39 tags read, none invented; the right drawing retrieved 12/12; 4/12 answers correct. Each run overwrites that file; copy it before an experiment.

## 2.10 Clean start

```
./services.sh status          # what is up
./services.sh start           # starts whatever is down
./services.sh check; echo "exit $?"
```

Expect every line `PASS` and `CLEAN START OK`, exit 0:

```
  PASS  python venv               3.12.15 arm64
  PASS  .env                      present, mode 600
  PASS  postgres                  ready on 127.0.0.1:5433
  PASS  pgvector extension        0.6.2
  PASS  data (fingerprint)        16 docs, 16 chunks, 8 superseded, 1 tables
  PASS  ollama                    Ollama 0.40.9
  PASS  model llama3.2:3b         present
  PASS  model nomic-embed-text    present
  PASS  smoke general               302 ms  "You're welcome! How can I assist you today?"
  PASS  smoke database              575 ms  'There are 2 brokers in rack-b. ...'
  PASS  smoke documents             748 ms  'A broker page should be triggered when the disk utilisation ...'
  PASS  api /health               {"documents":16,"chunks":16,"tables":["broker_inventory"]}
  PASS  web page /                HTTP 200
CLEAN START OK
```

With a service down the check names the fix (`FAIL postgres not ready: ./services.sh start`), prints `CLEAN START FAILED (n)` and exits 1. A `----` data line (for example after uploads or drawings) is information; `./recreate.sh` restores the baseline.

## 3. Scenario catalogue

Run them all: `.venv/bin/python scenarios.py --with-drawings` (about a minute; writes `results/scenarios_<time>.md`; exit 1 only on an unexpected failure). One at a time: `.venv/bin/python try.py "<question>"` with the settings shown. Results recorded on the M5 Pro on 2026-10-23 (`results/scenarios_20261023_113645.md`): 12 pass, 1 observed, 5 known weaknesses, 0 unexpected.

| Id | Question | Expected route and answer | M5 result | Shows |
|---|---|---|---|---|
| G1 | `Thanks!` | general; a short acknowledgement, no facts | pass, 0.3 s | the general agent never states facts |
| G2 | `What is the largest message size our brokers accept?` | documents with one rewrite; `not in the knowledge base` | pass, 2.4 s | grade finds nothing, one rewrite, then an explicit refusal |
| D1 | `At what consumer lag should operations raise an alert?` | documents; `5,000 messages` (not 10,000) | pass, 1.6 s | the normal document flow with a citation |
| D2 | `What port does BROKER-EAST-02 use for TLS traffic?` | documents; `9093` (not 9092) | pass, 1.5 s | an exact-string lookup; a control case for the version filter |
| D3 | `What replication factor and what default retention apply to a new standard production topic?` | documents; `3` and `72 hours` | known weakness: answers 3 only | multi-hop: the per-passage grader keeps one of the two passages |
| S1 | `Which brokers are above 85% disk usage?` | database; brokers 3 and 5 (87.2%, 91.8%) | pass, 1.1 s | the SQL agent with a WHERE from the question |
| S2 | `Which broker hosts the most topics, and how many?` | database; broker 5, 210 | known weakness: adds an unasked `WHERE status = 'healthy'` | the 3B model writing SQL (db-004) |
| S3 | `What is the total disk capacity across all brokers in GB?` | database; `16,000` | pass, 0.7 s | SUM; numbers copied exactly |
| S4 | `Delete all rows from the broker_inventory table.` | database; `only a single read-only SELECT is allowed` | pass, 0.6 s | the model writes a DELETE; the validator refuses it; the table is unchanged |
| V1 OFF | `VERSION_AWARE=0` + `At what disk utilisation should a broker page?` | documents | observed: `75%` | the old runbook (85%) is retrieved, but this corpus's current runbook names its former value, so the model still answers right; the retrieval ablation (2.6) is the proof |
| V1 ON | `VERSION_AWARE=1` + the same question | documents; `75%` | pass, 1.0 s | `WHERE d.superseded = false` before ranking |
| V2 | `Which team owns escalation for a SEV-1 Kafka incident?` | documents; `Streaming Platform team` (never `Platform Reliability`) | pass, 1.5 s | a version conflict with no number in it |

| Id | Question | Expected route and answer | M5 result | Shows |
|---|---|---|---|---|
| R1 | `What is the highest disk usage percentage and which broker has it?` | database; 91.8%, broker 5 | pass, 1.0 s | a rule in a runbook vs a measurement in a table: routed to the table |
| R2 | `How many minutes must consumer lag stay high before the alert fires?` | documents; `5 minutes` | known weakness (lean): `not in the knowledge base` | the 3B grader rejects a passage phrased differently ("sustained for 5 minutes") |
| R2+ | the same, with `GRADE_MODEL=qwen3:8b GRADE_MODE=escalate GRADE_MAX_TOKENS=600` | documents; `5 minutes` | pass, 6.9 s | the recommended profile: the second opinion keeps the passage |
| U1 | `Which cloud provider hosts the Kafka cluster?` | documents with rewrite; `not in the knowledge base` | pass, 2.5 s | unanswerable: refused, not guessed |
| M1 | `Which valve isolates the suction of pump P-101?` (after 2.9) | documents, drawing PID-001; `HV-101` | known weakness: `not in the knowledge base` | the drawing is retrieved; the answer step does not infer "suction side" |
| M2 | `Which instrument controls the flow into tank T-102, and through which valve?` | documents; `FIC-101`, `FCV-101` | known weakness: `not in the knowledge base` | the answer step does not chain controller and valve |
| W1 | (browser) `Which brokers are above 85% disk usage?` at http://127.0.0.1:8000/ | database; pills router > sql_agent > generate_sql | pass: router 271 ms, sql_agent 399 ms, generate_sql 362 ms; brokers 3 and 5 | the streaming contract: one server-sent event per step |

Where to see the evidence: `results/scenarios_*.json` (per scenario: route, path, time, answer, SQL), `traces.jsonl` (one line per question), `results/version_filter_off.json` and `_on.json` (which documents were retrieved, marked `OLD`), `results/m5_*.json` (per case: route, answer, pass/fail).

## 4. Fingerprints a correct rebuild must match

| What | Value |
|---|---|
| `golden/golden.jsonl` sha1 | `68ec09bdfcf99d62999ccfbe708f6f8ac3eaa4d3` (27 cases; frozen in commit db9af51 before any pipeline code) |
| `golden/golden_heldout.jsonl` sha1 | `02ddcfd613d4784f59bdf8ad20df971b8c9db899` (25 cases; frozen in 9510b6a before its first run) |
| `golden/amendments.jsonl` sha1 | `36a0e6e0d637a48ff43755fd579a1e7b9e80ecce` (one entry, doc-008, commit 45a0392) |
| corpus: `cat data/docs_versioned/*.md` piped into `shasum` | `c2b025608d03...`; `broker_inventory.csv` `c837aedfa3bd...` |
| database after 2.3 | `16 docs, 16 chunks, 8 superseded, 1 tables` |
| `b1_check.py` | `PASS`; pair similarity 0.963 to 0.990; old first 1/8 |
| `b1_eval.py` headline | OFF `99/192 (51.6%)`, ON `0/192 (0.0%)`; trap 2/12 to 12/12; control 20/20 both |
| `pid/make_pid.py` | regenerates byte for byte: `golden_pid.jsonl` sha1 `10fe1aa657131461df6d0b123de96fd2536c06ff`, `ground_truth.json` `1aaa5666aa0360fd237a175ad0188757e8e2c334`, the three drawings concatenated `d6dbc6b6174f...` |
| versions used | Python 3.12.15, langgraph 1.2.16, openai 3.37.0, psycopg 3.3.6, pgvector (python) 0.5.1, PostgreSQL 16.2 + pgvector 0.6.2 (pgserver 0.1.4), Ollama 0.40.9 |

`recreate.sh` checks the sha1s, the counts and the ablation headline and exits 1 on any mismatch.

## 5. How the evaluation is kept honest

1. The golden set was committed before the pipeline existed (db9af51), and the unseen set before its first run (9510b6a). Git order is the evidence.

2. A scorer fix (doc-008) went into `amendments.jsonl` in its own commit, never into the frozen file.

3. Golden questions are never pasted into prompts as examples.

4. Every prompt change is committed before it is measured, on the full set, by failing case id, not only by percentage.

5. Scores are reported over 3 repeated runs; the tuned set (92.6%) and the unseen set (80.0%) are reported side by side.

6. A scorer is never changed after seeing a result it would flip (llama3.1:8b's refusal in other words stays scored as a failure).

7. Latency is compared within one run (OFF and ON interleaved), never across runs.

## 6. Limits found on this machine

| | Evidence | What to do | |
|---|---|---|---|
| ader rejects phrased from the | R2; 4 of 5 unseen failures | the recommended profile (qwen3:8b second opinion): unseen 80% to 96% | |
| questions get r two facts | D3, h-doc-15 | split the question and retrieve per part (not built) | |
| odel sometimes masked SQL | S2, db-004 | a plausibility retry (not built) | |
| tions need over s | M1, M2; a bigger grader and a bigger answering model did not help | store connections as table rows (not built) | |
| Mac pauses a | a run stood still 16 minutes (`pmset -g log`) | `caffeinate -i` | |
| under-reports els | 0.8 GB shown, 5.0 GB resident | measure with `ps -axo rss,command | grep ollama` | |

## 7. The showcase site

`site/` is static: results, the flow, screenshots, the documents, and a replay of recorded runs from `site/data/replay.json` (rebuilt from `results/` by `site/build_replay.py`). It is updated locally; publishing (`./publish_site.sh`) and Vercel wait for the deployment stage.

## 8. Related documents

`docs/SOP_ragcap_build.pdf` (pre-work, preparation, implementation, testing, troubleshooting), `docs/RUNBOOK_ragcap_operations.pdf` (start/stop, clean start, routine tasks, backup, troubleshooting, models and memory, change control), `docs/INSTALL_AND_USE.md` (PDF `INSTALL_ragcap.pdf`: installing every tool by hand, step-by-step setup, daily use), `docs/APPLE_SILICON.md` (models: what is used and what to keep), `docs/build_pdfs.py` (regenerates the PDFs from this folder; needs `pip install reportlab`), `docs/ragcap_deck.pptx` (the slide deck, rebuilt by `docs/build_deck.js`).